

PROGRAMMING IN PYTHON AND CLEAN CODE PRINCIPLE

CAUIBC106T | Complete Detailed Notes and Practice Question Papers

Department of Computer Application | United University

Contents

- Unit I: Introduction to Clean Code and Python Fundamentals
- Unit II: Conditional and Control Statements
- Unit III: Functions and Data Structures
- Unit IV: Object-Oriented Programming in Python with Clean Coding
- Practice Question Paper 1: Units I and II
- Practice Question Paper 2: Units III and IV

How to Use These Notes

These notes are designed for semester-exam preparation and practical coding practice. Read the explanation, study the syntax, type each program yourself, predict the output, and then run it in Python. The examples use Python 3 syntax.

General clean-code habits: use meaningful names, keep functions small, avoid repetition, write comments only when they add meaning, validate input, handle errors clearly, and test both normal and boundary cases.

UNIT I: Introduction to Clean Code and Python Fundamentals

1. Introduction to Clean Code

Clean code is source code that is easy to read, understand, test, modify, and maintain. It is written for humans first and computers second. Code may work correctly and still be considered poor quality if it is confusing, repetitive, tightly coupled, or difficult to change.

Bad Code vs Clean Code

```
# Difficult to understand
x = 100
y = 20
z = x - (x * y / 100)
print(z)

# Cleaner version
original_price = 100
discount_percent = 20
final_price = original_price - (original_price * discount_percent / 100)
print(final_price)
```

Characteristics of clean code: meaningful names, consistent formatting, single responsibility, low duplication, simple logic, clear error handling, testability, and limited side effects.

2. Principles of Clean Code

- **Meaningful naming:** use names such as `total_marks` instead of `tm`.
- **Single Responsibility Principle:** one function or class should have one clear responsibility.
- **DRY:** Do Not Repeat Yourself; place repeated logic in a reusable function.
- **KISS:** Keep It Simple; avoid unnecessary complexity.
- **YAGNI:** You Aren't Gonna Need It; do not add features before they are required.
- **Consistency:** follow one naming, indentation, and formatting style.
- **Fail clearly:** report invalid inputs with useful error messages.
- **Testability:** separate input/output from core logic where possible.

3. Python: Definition and Uses

Python is a high-level, interpreted, general-purpose programming language known for readable syntax and a large standard library. It supports procedural, object-oriented, and functional programming styles.

Real-world uses: web development, data analysis, artificial intelligence, machine learning, automation, scripting, testing, scientific computing, and education.

4. History and Features of Python

- Python was created by Guido van Rossum and first released publicly in 1991.
- Python emphasizes readability and reduced syntax complexity.
- It is dynamically typed: a variable does not need a declared type.
- It is cross-platform and has extensive third-party libraries.

- Indentation is used to define code blocks.

5. Installation, IDE, and Execution

Install Python 3 from the official Python website or through your operating system's package manager. Common environments include IDLE, Visual Studio Code, PyCharm, Jupyter Notebook, and command-line terminals.

```
# Save as hello.py
print("Hello, Python!")

# Run from terminal
python hello.py
```

6. Python Syntax and Indentation

Python uses indentation, commonly four spaces, to group statements. Incorrect indentation can cause an `IndentationError`.

```
if True:
    print("This line belongs to the if block")
print("This line is outside the block")
```

7. Variables and Data Types

A variable is a name referring to a value. Assignment is performed using `=`. Python determines the type at runtime.

```
student_name = "Ayesha" # str
age = 20                # int
height = 1.65           # float
is_present = True       # bool
marks = [80, 75, 90]    # list
profile = {"id": 1}     # dict
nothing = None          # NoneType

print(type(student_name))
```

Main built-in types: int, float, complex, bool, str, list, tuple, set, dict, and `NoneType`.

8. Character Set, Tokens, and Comments

A token is the smallest meaningful unit in a program. Python tokens include keywords, identifiers, literals, operators, and delimiters.

- **Keywords:** if, else, for, while, def, class, return, True, False, None.
- **Identifiers:** names given to variables, functions, classes, and objects.
- **Literals:** fixed values such as 10, 3.14, 'hello', and True.
- **Comments:** text ignored by the interpreter, written using `#`.

```
# This is a single-line comment
name = "Kiran" # Inline comment
```

9. Input, Output, eval(), and f-Strings

`input()` reads text from the user and returns a string. Convert the result when a number is required. `print()` displays output. `eval()` evaluates a Python expression and should not be used with untrusted input because it can execute harmful code.

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
print("Name:", name)
print("Age:", age)

a = 10
b = 20
print(f"Sum of {a} and {b} is {a+b}")
```

f-strings allow expressions inside braces and are usually clearer than manual string concatenation.

10. Operators and Expressions

- **Arithmetic:** +, -, *, /, //, %, **

- **Comparison:** ==, !=, >, <, >=, <=
- **Logical:** and, or, not
- **Assignment:** =, +=, -=, *=, /=, //=, %=, **=
- **Membership:** in, not in
- **Identity:** is, is not
- **Bitwise:** &, |, ^, ~, <<, >>

```
a, b = 7, 2
print(a + b)    # 9
print(a / b)    # 3.5
print(a // b)   # 3
print(a % b)    # 1
print(a ** b)   # 49
print(a > b and b > 0) # True
```

11. Type Conversion

Type conversion changes a value from one data type to another. Explicit conversion functions include `int()`, `float()`, `str()`, `bool()`, `list()`, `tuple()`, and `set()`.

```
number_text = "25"
number = int(number_text)
price = float("99.50")
message = str(100)
values = list((1, 2, 3))
print(number + 5)
```

Important: `int('hello')` raises `ValueError`. Always validate user input when appropriate.

UNIT II: Conditional and Control Statements

1. Decision Making

Decision-making statements execute different blocks based on a condition. A condition evaluates to `True` or `False`.

```
marks = 72

if marks >= 40:
    print("Pass")
else:
    print("Fail")
```

2. if, elif, and else

```
marks = int(input("Enter marks: "))

if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 60:
    grade = "C"
elif marks >= 40:
    grade = "D"
else:
    grade = "F"

print(f"Grade: {grade}")
```

The conditions are checked from top to bottom. Once a condition is true, its block executes and the remaining `elif/else` blocks are skipped.

3. Nested if

```
age = 22
has_id = True

if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
```

```

        print("ID required")
    else:
        print("Entry not allowed")

```

4. Conditional Expressions

```

age = 19
status = "Adult" if age >= 18 else "Minor"
print(status)

```

5. Boolean Type and Operators

The bool type has two values: True and False. In Python, empty strings, zero, empty collections, and None are generally falsey; most other values are truthy.

```

username = ""
if not username:
    print("Username is missing")

print(True and False) # False
print(True or False)  # True
print(not True)        # False

```

6. String Operators

```

text = "Python"
print(text + " Programming")
print(text * 2)
print("Py" in text)
print(text[0])
print(text[1:4])

```

7. while Loop

A while loop repeats while its condition remains true. Update the loop variable to avoid an infinite loop.

```

count = 1
while count <= 5:
    print(count)
    count += 1

```

8. for Loop and range()

```

for number in range(1, 6):
    print(number)

for index in range(0, 10, 2):
    print(index)

```

range(start, stop, step) generates values from start up to but not including stop. The default start is 0 and default step is 1.

9. Nested Loops

```

for row in range(1, 4):
    for column in range(1, 4):
        print(" ", end=" ")
    print()

```

10. break, continue, and pass

```

for number in range(1, 6):
    if number == 3:
        continue
    if number == 5:
        break
    print(number)

def future_feature():
    pass

```

break terminates the nearest loop. continue skips the current iteration. pass is a placeholder that performs no action.

11. Float Representation

Floating-point values are stored approximately in binary. Therefore, some decimal calculations may not be exact.

```
print(0.1 + 0.2) # may display 0.30000000000000004
print(round(0.1 + 0.2, 2)) # 0.3
```

For financial calculations requiring exact decimal behavior, consider the decimal module rather than relying only on float.

UNIT III: Functions and Data Structures

1. Functions: Syntax and Basics

A function is a reusable block of code that performs a specific task. It reduces duplication and improves organization.

```
def greet():
    print("Welcome to Python")

greet()
```

2. Parameters, Arguments, and Return

```
def add_numbers(first, second):
    return first + second

result = add_numbers(10, 20)
print(result)
```

A parameter is a name in the function definition. An argument is the actual value passed during the call. return sends a value back to the caller and ends the function.

3. Default, Keyword, and Variable-Length Arguments

```
def introduce(name, course="Python"):
    print(f"{name} studies {course}")

introduce("Sara")
introduce(name="Ali", course="Clean Code")

def total(*numbers):
    return sum(numbers)

print(total(1, 2, 3, 4))
```

4. Local and Global Scope

```
message = "global"

def show_message():
    message = "local"
    print(message)

show_message()
print(message)
```

A local variable exists inside its function. A global variable is defined outside functions. Use global sparingly because excessive global state makes testing and maintenance harder.

5. Recursive Functions

Recursion occurs when a function calls itself. A base case is required to stop recursion.

```
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))
```

6. Lambda, map, filter, and reduce

```
from functools import reduce

numbers = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x * x, numbers))
```

```
evens = list(filter(lambda x: x % 2 == 0, numbers))
product = reduce(lambda a, b: a * b, numbers)

print(squares)
print(evens)
print(product)
```

Use lambda for short expressions. For complex logic, a named def function is usually more readable.

7. Strings and Built-in String Functions

```
text = " Python Programming "
print(text.strip())
print(text.lower())
print(text.upper())
print(text.replace("Python", "Java"))
print(text.split())
print("-".join(["clean", "code"]))
print(text.startswith(" Py"))
```

Strings are immutable, meaning string methods return new strings rather than changing the original string in place.

8. String Traversal and Operators

```
word = "CODE"
for character in word:
    print(character)

print("C" in word)
print(word[0:2])
print(word[::-1])
```

9. Lists

A list is an ordered, mutable collection. It can contain repeated values and different data types.

```
numbers = [10, 20, 30]
numbers.append(40)
numbers.insert(1, 15)
numbers.remove(20)
last = numbers.pop()
numbers.sort()
print(numbers)
print(last)
```

Common list functions/methods: len(), append(), extend(), insert(), remove(), pop(), clear(), index(), count(), sort(), reverse(), copy().

10. Tuples

A tuple is an ordered, immutable collection. Tuples are useful for fixed groups of values.

```
coordinates = (10, 20)
x, y = coordinates
print(x, y)
```

11. Dictionaries

A dictionary stores key-value pairs. Keys must be hashable and unique.

```
student = {"name": "Riya", "age": 20, "marks": 88}
print(student["name"])
print(student.get("grade", "Not available"))
student["marks"] = 91
student["city"] = "Lucknow"

for key, value in student.items():
    print(key, value)
```

Common dictionary methods: keys(), values(), items(), get(), update(), pop(), popitem(), setdefault(), clear().

12. Sets

A set is an unordered collection of unique elements. It is useful for removing duplicates and performing mathematical set operations.

```

a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(a | b) # union
print(a & b) # intersection
print(a - b) # difference
print(a ^ b) # symmetric difference

```

13. List, Tuple, Set, and Dictionary Comparison

Type	Ordered	Mutable	Duplicates	Main use
List	Yes	Yes	Yes	Dynamic sequence
Tuple	Yes	No	Yes	Fixed sequence
Set	No guaranteed order	Yes	No	Unique values
Dictionary	Insertion order preserved	Yes	Keys unique	Key-value lookup

UNIT IV: OOP in Python with Clean Coding

1. Object-Oriented Programming

Object-Oriented Programming (OOP) organizes programs using classes and objects. A class is a blueprint; an object is an instance created from that class.

```

class Student:
    def display(self):
        print("This is a student")

student1 = Student()
student1.display()

```

2. self and Constructor Methods

self refers to the current object. `__init__()` is commonly used as a constructor-like initializer to assign object attributes.

```

class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def show(self):
        print(f"{self.name}: {self.marks}")

student = Student("Aman", 85)
student.show()

```

3. Encapsulation

Encapsulation groups data and methods together and controls how internal state is accessed. Python uses naming conventions such as `_name` and name-mangling with `__name`; these are not absolute private access controls.

```

class BankAccount:
    def __init__(self, balance):
        self._balance = balance

    def deposit(self, amount):
        if amount > 0:
            self._balance += amount

    def get_balance(self):
        return self._balance

```

4. Abstraction

Abstraction exposes essential behavior while hiding implementation details. The `abc` module supports abstract base classes.

```

from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):

```

```

        pass

class Square(Shape):
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side * self.side

print(Square(4).area())

```

5. Inheritance

Inheritance allows a child class to reuse or extend behavior from a parent class.

```

class Animal:
    def speak(self):
        print("Animal sound")

class Dog(Animal):
    def speak(self):
        print("Bark")

dog = Dog()
dog.speak()

```

6. Polymorphism

Polymorphism means the same interface or method name can work with different object types.

```

class Cat:
    def speak(self):
        return "Meow"

class Dog:
    def speak(self):
        return "Bark"

for animal in [Cat(), Dog()]:
    print(animal.speak())

```

7. SOLID Principles

- **S — Single Responsibility:** a class should have one reason to change.
- **O — Open/Closed:** software should be open for extension but closed for modification.
- **L — Liskov Substitution:** child objects should be usable wherever parent objects are expected.
- **I — Interface Segregation:** prefer small, focused interfaces over large general ones.
- **D — Dependency Inversion:** depend on abstractions rather than concrete implementations.

8. Refactoring Techniques

Refactoring improves internal code structure without changing externally observable behavior.

- Rename unclear variables and functions.
- Extract repeated logic into functions.
- Split long functions into smaller units.
- Replace deeply nested conditions with guard clauses.
- Remove unused variables and unreachable code.
- Separate business logic from input/output.
- Use constants instead of repeated magic numbers.

9. DRY and Avoiding Code Smells

DRY means Do Not Repeat Yourself. A code smell is a warning sign that code may have a design or maintenance problem.

- **Long method:** a function performs too many tasks.
- **Large class:** a class has too many responsibilities.
- **Duplicate code:** the same logic appears in multiple places.
- **Magic numbers:** unexplained numeric values appear directly in logic.
- **Deep nesting:** many levels of if/loop nesting make code hard to follow.
- **God object:** one object controls unrelated parts of the application.

10. Clean OOP Example

```
class GradeCalculator:
    PASS_MARKS = 40

    def __init__(self, marks):
        self.marks = marks

    def is_pass(self):
        return self.marks >= self.PASS_MARKS

    def result(self):
        return "Pass" if self.is_pass() else "Fail"

calculator = GradeCalculator(72)
print(calculator.result())
```

This example uses a meaningful class name, a constant, small methods, and a clear responsibility.

Testing and Debugging Essentials

Testing checks whether a program behaves as expected. Debugging is the process of finding and correcting defects.

- Test normal inputs, boundary inputs, invalid inputs, and empty inputs.
- Use print statements or a debugger to inspect values.
- Read the traceback from the bottom upward to identify the error type and line.
- Common errors include `SyntaxError`, `NameError`, `TypeError`, `ValueError`, `IndexError`, `KeyError`, and `ZeroDivisionError`.

```
def divide(a, b):
    if b == 0:
        raise ValueError("Denominator cannot be zero")
    return a / b

try:
    print(divide(10, 0))
except ValueError as error:
    print(f"Error: {error}")
```